Міністерство освіти і науки України
Національний технічний університет
«Дніпровська політехніка»

ЗВІТ

про виконання лабораторної роботи

з дисципліни
«Поглиблене програмування мовою Java»

Виконав(-ла): ст. гр. 121-22-2
Макаров Данило Євгенович

Прийняв(-ла): доц. Мінєєв О.С.

Дніпро
2026

1. Загальний опис проекту

PriceTracker — веб-застосунок на Spring Boot, що дозволяє користувачам відстежувати ціни товарів з інтернет-магазину ks-market.com.ua. Застосунок автоматично парсить ціну та наявність товару, конвертує ціну в USD за актуальним курсом PrivatBank та зберігає повну історію зміни цін у базі даних.

1.1 Основні функції

- Додавання товарів за URL-адресою з ks-market.com.ua
- Автоматичне парсингування назви, зображення, ціни та наявності товару
- Конвертація ціни UAH → USD з кешуванням курсу (1 година)
- Зберігання повної історії змін цін
- Soft-delete: видалені товари не зникають з БД, а лише позначаються як видалені
- REST API для управління товарами
- Сортування товарів за ціною (зростання / спадання)

1.2 Підтримуваний магазин

Наразі підтримується виключно домен: https://ks-market.com.ua/ua/...
URL-адреси з інших доменів будуть відхилені з помилкою валідації.

2. Стек технологій та бібліотеки

Технологія / Бібліотека	Призначення	Коментар
Java 21	Мова програмування	LTS-версія з virtual threads, records, pattern matching
Spring Boot 3.x	Основний фреймворк	Автоконфігурація, вбудований Tomcat, DI-контейнер
Spring MVC	Веб-шар	Обробка HTTP-запитів, контролери, маршрутизація
Spring Data JPA	Шар доступу до даних	Репозиторії, генерація SQL, Hibernate ORM
Hibernate	ORM	Маппінг Java-об'єктів на таблиці БД
MySQL / PostgreSQL	Реляційна СУБД	Зберігання товарів, цін та курсів валют
Thymeleaf	Шаблонізатор	Серверний рендеринг HTML-сторінок
Jsoup 1.x	HTML-парсер	Завантаження та парсинг сторінок ks-market
Jackson	JSON-серіалізація	Серіалізація DTO, парсинг відповіді PrivatBank API
RestTemplate	HTTP-клієнт	Запити до зовнішніх API (курси валют PrivatBank)
Maven	Система збірки	Управління залежностями та збіркою проекту

2.1 Чому саме ці технології?

Spring Boot

Дозволяє швидко розгорнути production-ready застосунок без ручного налаштування XML. Вбудований Tomcat усуває потребу у зовнішньому сервері. Автоконфігурація мінімізує boilerplate-код.

Spring Data JPA + Hibernate

Репозиторії генерують SQL-запити автоматично за назвою методу (findByIsDeletedFalse, findTopByCurrencyCodeOrderByFetchedAtDesc тощо). Це значно скорочує кількість ручного SQL та спрощує підтримку.

Jsoup

Зручна бібліотека для парсингу HTML. Підтримує CSS-селектори (selectFirst, select), автоматично обробляє кодування та дозволяє емулювати браузер через заголовок User-Agent.

RestTemplate

Стандартний синхронний HTTP-клієнт Spring. Використовується для отримання курсів валют з PrivatBank API. Для нових проектів рекомендується WebClient, але RestTemplate простіший для синхронних сценаріїв.

Soft-delete патерн

Замість фізичного видалення рядків з БД встановлюється прапорець isDeleted = true. Це зберігає історію, дозволяє відновлювати товари та уникає порушення зовнішніх ключів у таблиці price_history.

3. Архітектура проекту

3.1 Пакетна структура

```
ua.edu.example.pricetracker
├── PriceTrackerApplication.java   // Точка входу
├── config/
│   └── AppConfig.java             // Spring конфігурація (RestTemplate bean)
├── controller/
│   ├── HomeController.java        // Сторінка /
│   └── ProductController.java     // REST + MVC /products
├── dto/
│   ├── ProductCreateRequest.java  // Тіло POST /api/products
│   ├── ProductDTO.java            // Відповідь API з даними товару
│   └── ScrapedProductData.java    // Результат парсингу
├── model/
│   ├── Product.java               // JPA-сутність: таблиця products
│   ├── PriceHistory.java          // JPA-сутність: таблиця price_history
│   └── ExchangeRate.java          // JPA-сутність: таблиця exchange_rates
├── repository/
│   ├── ProductRepository.java
│   ├── PriceHistoryRepository.java
│   └── ExchangeRateRepository.java
└── service/
    ├── ProductService.java        // Основна бізнес-логіка
    ├── ScraperService.java        // Парсинг ks-market
    ├── ExchangeRateService.java   // Курс USD/UAH (PrivatBank API)
    └── PriceHistoryService.java   // Запис змін цін
```

3.2 Шари застосунку

Шар	Опис
Presentation Layer	Controller-класи: обробляють HTTP-запити, повертають View або JSON
Service Layer	Бізнес-логіка: ProductService, ScraperService, ExchangeRateService, PriceHistoryService
Repository Layer	Доступ до БД через Spring Data JPA інтерфейси
Model Layer	JPA-сутності: Product, PriceHistory, ExchangeRate
DTO Layer	Об'єкти передачі даних між шарами (ProductDTO, ScrapedProductData)

3.3 Потік даних при додаванні товару

1. Клієнт надсилає POST /api/products з тілом { "url": "https://ks-market.com.ua/ua/..." }
2. ProductController.addProduct() приймає запит, викликає ProductService.addProduct(url)
3. ProductService перевіряє: чи товар вже є в БД (за URL)
4. ScraperService.scrapeProduct(url) — завантажує HTML сторінки, парсить name, imageUrl, priceUah, inStock
5. ExchangeRateService.convertUahToUsd(priceUah) — повертає ціну в USD (з кешем 1 год)
6. Product зберігається в БД через ProductRepository.save()
7. PriceHistoryService.recordPrice() — записує перший запис в price_history
8. Повертається ProductDTO клієнту як JSON

4. Схема бази даних

4.1 Таблиця: products

Поле	Тип / Опис
id	BIGINT, PK, AUTO_INCREMENT
name	VARCHAR(500), NOT NULL — назва товару
product_url	VARCHAR(1000), NOT NULL, UNIQUE — URL товару
image_url	VARCHAR(1000) — URL зображення
price_uah	DECIMAL(10,2), NOT NULL — ціна в гривнях
price_usd	DECIMAL(10,2), NOT NULL — ціна в доларах
in_stock	BOOLEAN, NOT NULL — наявність
is_deleted	BOOLEAN, NOT NULL, DEFAULT false — soft-delete
created_at	DATETIME, NOT NULL — дата додавання
updated_at	DATETIME, NOT NULL — дата останнього оновлення

4.2 Таблиця: price_history

Поле	Тип / Опис
id	BIGINT, PK, AUTO_INCREMENT
product_id	BIGINT, FK → products(id), NOT NULL
price_uah	DECIMAL(10,2), NOT NULL
price_usd	DECIMAL(10,2), NOT NULL
recorded_at	DATETIME, NOT NULL — дата запису

4.3 Таблиця: exchange_rates

Поле	Тип / Опис
id	BIGINT, PK, AUTO_INCREMENT
currency_code	VARCHAR(3), NOT NULL — наприклад 'USD'
buy_rate	DECIMAL(10,4), NOT NULL — курс купівлі
sell_rate	DECIMAL(10,4), NOT NULL — курс продажу

Поле	Тип / Опис
fetched_at	DATETIME, NOT NULL — час отримання курсу

4.4 Зв'язки між таблицями

products → price_history: один до багатьох (One-to-Many). Один товар може мати багато записів в price_history. Зв'язок через product_id (ManyToOne з FetchType.LAZY).

5. JPA-моделі (Model Layer)

5.1 Product.java

Основна сутність, що відповідає таблиці products. Анотації @PrePersist та @PreUpdate автоматично виставляють часові мітки:

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_url", nullable = false, unique = true, length = 1000)
    private String productUrl;

    @Column(name = "is_deleted", nullable = false)
    private boolean isDeleted = false;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

5.2 PriceHistory.java — FetchType.LAZY

Зберігає снепшот ціни на конкретний момент часу. Зв'язок з товаром оголошено через @ManyToOne(fetch = FetchType.LAZY):

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "product_id", nullable = false)
private Product product;
```

Що таке FetchType.LAZY і чому саме він?

У JPA є два режими завантаження пов'язаних сутностей:

- **EAGER (жадібне завантаження):** Hibernate одразу при завантаженні PriceHistory виконує додатковий SQL-запит і завантажує пов'язаний Product. Навіть якщо об'єкт Product вам не потрібен.
- **LAZY (ліниве завантаження):** Hibernate НЕ завантажує Product автоматично. Замість реального об'єкта підставляється проксі-об'єкт. Реальний SQL виконується лише тоді, коли хтось вперше викликає history.getProduct().

У нашому проекті FetchType.LAZY обраний з таких причин:

- Продуктивність: у таблиці price_history може бути тисячі рядків. Якби кожен запит "дай мені всю історію" автоматично підтягував Product для кожного запису — це N+1 запитів до БД.
- Непотрібність: сервіс PriceHistoryService лише зберігає та повертає записи цін. Дані про сам товар при цьому не потрібні.
- Контроль: при потребі отримати товар можна завжди явно через history.getProduct() всередині активної транзакції.

Таким чином LAZY — це стандартна рекомендована практика для всіх @ManyToOne зв'язків у JPA. EAGER варто використовувати лише тоді, коли пов'язана сутність гарантовано завжди потрібна разом з основною.

5.3 ExchangeRate.java

Кешований курс валют від PrivatBank. Зберігаються обидва курси — buy та sell. При конвертації використовується sellRate (курс продажу банком).

6. Репозиторії (Repository Layer)

Всі репозиторії розширюють JpaRepository<Entity, Long>, що автоматично надає методи save(), findById(), findAll(), delete() тощо.

6.1 ProductRepository

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Знайти товар за URL (для перевірки дублікатів) → Optional
    Optional<Product> findByProductUrl(String productUrl);

    // Всі активні товари (без видалених)
    List<Product> findByIsDeletedFalse();

    // З сортуванням за ціною
    List<Product> findByIsDeletedFalseOrderByPriceUahAsc();
    List<Product> findByIsDeletedFalseOrderByPriceUahDesc();
}
```

6.2 PriceHistoryRepository

```
@Repository
public interface PriceHistoryRepository extends JpaRepository<PriceHistory, Long>
{

    // Історія цін для конкретного товару, від новіших до старіших
    List<PriceHistory> findByProductIdOrderByRecordedAtDesc(Long productId);
}
```

6.3 ExchangeRateRepository

```
@Repository
public interface ExchangeRateRepository extends JpaRepository<ExchangeRate, Long>
{

    // Найсвіжіший запис курсу для валюти → Optional
    Optional<ExchangeRate> findTopByCurrencyCodeOrderByFetchedAtDesc(String
currencyCode);
}
```

6.4 Чому деякі методи повертають Optional?

Optional<T> — це контейнерний тип Java (з'явився у Java 8), який явно сигналізує: "цей метод може не знайти результат". Він є альтернативою поверненню null.

У проекті Optional використовується у двох місцях:

- **findByProductUrl(String url) → Optional<Product>:** Товар з таким URL може бути в БД, а може й не бути. Optional змушує викликаючий код явно обробити обидва випадки: existing.isPresent() / existing.get().
- **findTopByCurrencyCodeOrderByFetchedAtDesc(String code) → Optional<ExchangeRate>:** Кешованого курсу може ще не бути в БД (перший запуск). Optional сигналізує про це без виключення.

Переваги Optional перед поверненням null:

- Компілятор та IDE підказують, що результат може бути відсутнім — неможливо «забути» перевірку
- Код стає самодокументованим: Optional у підписі методу відразу пояснює поведінку
- Уникнення NullPointerException — найпоширенішої помилки в Java
- Зручний API: isPresent(), get(), orElse(), orElseThrow(), ifPresent() тощо

Для методів findById() Optional повертається автоматично самим JpaRepository — це стандартна поведінка Spring Data.

7. Сервісний шар (Service Layer)

7.1 ProductService

Центральний сервіс, що координує всю бізнес-логіку. Залежності: ProductRepository, ScraperService, ExchangeRateService, PriceHistoryService.

7.1.1 addProduct(String url)

Повний алгоритм додавання нового товару:

```
public ProductDTO addProduct(String url) {
    if (url != null) url = url.trim();

    // 1. Перевірка дублікату
    Optional<Product> existing = productRepository.findByProductUrl(url);
    if (existing.isPresent()) {
        Product p = existing.get();
        if (!p.isDeleted()) {
            throw new IllegalArgumentException("Цей товар вже додано до списку");
        }
        // Відновлення soft-deleted товару
        p.setDeleted(false);
        return refreshProductInternal(p);
    }

    // 2. Парсинг сторінки
    ScrapedProductData scraped = scraperService.scrapeProduct(url);

    // 3. Конвертація ціни
    BigDecimal priceUsd =
exchangeRateService.convertUahToUsd(scraped.getPriceUah());

    // 4. Збереження в БД
    Product product = new Product();
    // ... встановлення полів ...
    product = productRepository.save(product);

    // 5. Початковий запис в історію цін
    priceHistoryService.recordPrice(product, scraped.getPriceUah(), priceUsd);

    return toDTO(product);
}
```

7.1.2 refreshProduct(Long id)

Оновлення даних наявного товару. Внутрішній метод refreshProductInternal перевіряє, чи змінилась ціна, і лише тоді записує новий рядок в price_history — це запобігає дублюванню однакових записів:

```
private ProductDTO refreshProductInternal(Product product) {
    ScrapedProductData scraped =
scraperService.scrapeProduct(product.getProductUrl());
    BigDecimal priceUsd =
exchangeRateService.convertUahToUsd(scraped.getPriceUah());

    boolean priceChanged = product.getPriceUah() == null ||
        product.getPriceUah().compareTo(scraped.getPriceUah()) != 0 ||
        product.getPriceUsd() == null ||
        product.getPriceUsd().compareTo(priceUsd) != 0;

    // ... оновлення полів ...
    product = productRepository.save(product);

    if (priceChanged) {
        priceHistoryService.recordPrice(product, scraped.getPriceUah(), priceUsd);
    }
    return toDTO(product);
}
```

7.1.3 deleteProduct(Long id) — Soft-Delete

Замість видалення рядка з БД, метод лише встановлює прапорець isDeleted = true. Всі запити на отримання списку товарів фільтрують такі записи:

```
public void deleteProduct(Long id) {
    Product product = productRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Товар не знайдено"));
    product.setDeleted(true);
    productRepository.save(product);
}
```

7.2 ScraperService

Відповідає за завантаження та парсинг HTML-сторінок ks-market.com.ua. Використовує бібліотеку Jsoup.

7.2.1 Валідація URL

```
private void validateUrl(String url) {
    if (url == null || url.isBlank()) {
        throw new IllegalArgumentException("Будь ласка, введіть коректне
посилання");
    }
    if (!url.startsWith("https://ks-market.com.ua/ua/")) {
        throw new IllegalArgumentException(
            "Підтримуються лише посилання формату https://ks-
market.com.ua/ua/...");
    }
}
```

7.2.2 CSS-селектори для парсингу

Дані	CSS-селектор / Логіка
Назва товару	h1.inbreadcrumb — заголовок H1 з класом inbreadcrumb
Зображення	img#mainImage — тег img з id="mainImage", атрибут src
Ціна (основний)	meta[itemprop=price] — мета-тег з атрибутом content
Ціна (запасний)	span.update_price_options — span з класом update_price_options
Наявність	div.nalich — текст містить "є в наявності" або "есть в наличии"

7.3 ExchangeRateService

Отримує курс USD/UAH від PrivatBank API з кешуванням у БД. Кеш діє 1 годину — якщо запис в БД не старіший за 1 годину, нова HTTP-відповідь не надсилається.

7.3.1 Алгоритм отримання курсу

```
public BigDecimal getUsdSellRate() {
    Optional<ExchangeRate> cached = exchangeRateRepository
        .findTopByCurrencyCodeOrderByFetchedAtDesc("USD");

    if (cached.isPresent() &&
        cached.get().getFetchedAt().isAfter(LocalDateTime.now().minusHours(1))) {
        return cached.get().getSellRate(); // повертаємо кешований курс
    }

    return fetchAndCacheRate();
}
```

7.3.2 PrivatBank API

Endpoint: https://api.privatbank.ua/p24api/pubinfo?exchange&coursid=5

Відповідь — масив JSON з полями ccy, base_ccy, buy, sale. Fallback: якщо API недоступне, сервіс повертає останній збережений курс незалежно від давності.

7.4 PriceHistoryService

```
public void recordPrice(Product product, BigDecimal priceUah, BigDecimal priceUsd)
{
    PriceHistory history = new PriceHistory();
    history.setProduct(product);
    history.setPriceUah(priceUah);
    history.setPriceUsd(priceUsd);
    priceHistoryRepository.save(history);
}
```

8. Контролери (Controller Layer)

8.1 AppConfig.java

Реєструє RestTemplate як Spring Bean для впровадження залежностей у сервіси:

```
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

8.2 HomeController.java

Обробляє головну сторінку /. Передає в Thymeleaf-шаблон список активних товарів:

```
@Controller
public class HomeController {
    @GetMapping("/")
    public String index(Model model) {
        model.addAttribute("products", productService.getProducts(null));
        return "index";
    }
}
```

8.3 ProductController.java — REST API

Поєднує MVC та REST. Всі REST-методи анотовані @ResponseBody і повертають JSON.

8.3.1 REST-ендпоінти

Ендпоінт	Опис
GET /products	MVC: сторінка управління товарами з параметром ?sort=asc|desc
GET /api/products	REST: JSON-список активних товарів (опц. ?sort=asc|desc)

Ендпоінт	Опис
POST /api/products	REST: додати товар. Тіло: { "url": "https://..." }
PUT /api/products/{id}/refresh	REST: оновити ціну товару за id
DELETE /api/products/{id}	REST: soft-delete товару за id

8.3.2 Формат відповіді API

```
// Успішна відповідь:
{ "success": true,  "message": "Товар успішно додано", "product": {
...ProductDTO... } }

// Помилка:
{ "success": false, "message": "Опис помилки" }
```

8.3.3 Обробка виключень

IllegalArgumentException (валідація URL, дублікат товару) → HTTP 400 Bad Request.

RuntimeException (парсинг, БД, API) → HTTP 500 Internal Server Error.

9. Data Transfer Objects (DTO)

9.1 ProductCreateRequest

Тіло POST-запиту при додаванні товару. Містить єдине поле url.

9.2 ProductDTO

Об'єкт відповіді API. Не містить службових полів моделі (isDeleted, createdAt). Jackson серіалізує його в JSON автоматично.

Поле	Тип / Призначення
id	Long — ідентифікатор
name	String — назва товару
productUrl	String — URL сторінки
imageUrl	String — URL зображення
priceUah	BigDecimal — ціна в гривнях
priceUsd	BigDecimal — ціна в доларах
inStock	boolean — наявність
updatedAt	LocalDateTime — дата оновлення

9.3 ScrapedProductData

Внутрішній DTO для передачі результатів парсингу між ScraperService та ProductService. Не виходить за межі сервісного шару.

10. Пайплайни операцій

10.1 POST /api/products — додавання товару

```
HTTP POST /api/products
  Body: { "url": "https://ks-market.com.ua/ua/product-slug" }
    │
    ▼
ProductController.addProduct(request)
    │
    ▼
ProductService.addProduct(url)
    ├── url.trim()
    ├── productRepository.findByProductUrl(url)  ← повертає Optional
    │     ├── [знайдено, isDeleted=false] → throw IllegalArgumentException
    │     ├── [знайдено, isDeleted=true]  → відновити + refreshProductInternal()
    │     └── [не знайдено] → продовжити
    │
    ├── ScraperService.scrapeProduct(url)
    │     ├── validateUrl(url)
    │     ├── Jsoup.connect(url).userAgent(...).get()
    │     └── повертає ScrapedProductData
    │
    ├── ExchangeRateService.convertUahToUsd(priceUah)
    │     ├── exchangeRateRepository.findTop...  ← повертає Optional
    │     ├── [кеш актуальний] → повернути sellRate
    │     └── [кеш застарів] → GET PrivatBank API → зберегти → повернути
    │
    ├── productRepository.save(product)
    ├── priceHistoryService.recordPrice(...)
    └── повернути ProductDTO
    │
    ▼
HTTP 200 OK: { success: true, product: ProductDTO }
```

10.2 PUT /api/products/{id}/refresh

```
HTTP PUT /api/products/{id}/refresh
    ▼
ProductService.refreshProduct(id)
    ├── findById(id)  ← Optional → orElseThrow
    ├── [isDeleted=true] → throw RuntimeException
    └── refreshProductInternal(product)
          ├── ScraperService.scrapeProduct(url)
          ├── ExchangeRateService.convertUahToUsd(priceUah)
          ├── порівняння старої та нової ціни
          ├── productRepository.save(product)
          ├── [ціна змінилась] → priceHistoryService.recordPrice()
          └── повернути ProductDTO
```

10.3 DELETE /api/products/{id}

```
HTTP DELETE /api/products/{id}
    ▼
ProductService.deleteProduct(id)
    ├── findById(id) → orElseThrow
    ├── product.setDeleted(true)
    └── productRepository.save(product)
    ▼
HTTP 200 OK: { success: true, message: "Товар видалено" }
```

10.4 Кешування курсу валют

```
ExchangeRateService.getUsdSellRate()
    │
    ├── findTopByCurrencyCodeOrderByFetchedAtDesc("USD")  ← Optional
    │
    ├── [є запис && fetchedAt > now()-1h] → повернути sellRate з БД
    │
    └── [немає або застарів]
          ├── GET https://api.privatbank.ua/p24api/pubinfo?exchange&coursid=5
          ├── знайти { ccy: "USD" } у масиві відповіді
          ├── зберегти новий ExchangeRate
          ├── повернути sale (курс продажу)
          └── [виняток] → fallback: будь-який збережений курс
```

11. Обробка помилок

Ситуація	Місце виникнення	Наслідок
Невалідний URL	ScraperService.validateUrl()	IllegalArgumentException → HTTP 400
Товар вже доданий	ProductService.addProduct()	IllegalArgumentException → HTTP 400
Товар не знайдено	refreshProduct() / delete()	RuntimeException → HTTP 500
Товар позначений видаленим	ProductService.refreshProduct()	RuntimeException → HTTP 500
Таймаут / недоступний сайт	ScraperService.scrapeProduct()	RuntimeException (IOException) → HTTP 500
Не вдалося спарсити дані	ScraperService.scrapeProduct()	RuntimeException → HTTP 500
PrivatBank API недоступний	ExchangeRateService	Fallback: старий курс або RuntimeException

12. Точка входу застосунку

```
@SpringBootApplication
public class PriceTrackerApplication {
    public static void main(String[] args) {
        SpringApplication.run(PriceTrackerApplication.class, args);
    }
}
```

Анотація @SpringBootApplication еквівалентна поєднанню: @Configuration, @EnableAutoConfiguration, @ComponentScan. Запускає автовиявлення всіх бінів у пакеті ua.edu.example.pricetracker та підпакетах.

13. Конфігурація та розгортання

13.1 application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/pricetracker?useSSL=false
spring.datasource.username=root
spring.datasource.password=your_password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

13.2 Збірка та запуск

```
# Збірка (Maven)
mvn clean package -DskipTests

# Запуск JAR
java -jar target/pricetracker-0.0.1-SNAPSHOT.jar

# Або через Maven
mvn spring-boot:run
```

13.3 Вимоги до середовища

- Java 21+ (JDK)
- MySQL 8.0+ або PostgreSQL 14+
- Доступ до інтернету (ks-market.com.ua та api.privatbank.ua)
- Maven 3.8+

14. Глосарій

Термін	Визначення
Soft-delete	Логічне видалення: запис позначається isDeleted=true замість реального DELETE
DTO	Data Transfer Object — об'єкт для передачі даних між шарами або по мережі
FetchType.LAZY	Ліниве завантаження: пов'язана сутність JPA завантажується лише при явному зверненні до неї
FetchType.EAGER	Жадібне завантаження: пов'язана сутність завантажується автоматично разом з основною
Optional<T>	Контейнер Java, що явно вказує на можливу відсутність значення (альтернатива null)
Scraping	Автоматизоване отримання даних з веб-сторінки шляхом парсингу HTML
JPA	Java Persistence API — специфікація для ORM у Java
ORM	Object-Relational Mapping — маппінг Java-об'єктів на таблиці БД
REST API	HTTP API з ресурсами та стандартними методами (GET/POST/PUT/DELETE)
Bean	Об'єкт, керований Spring IoC-контейнером
sellRate	Курс продажу: скільки UAH потрібно заплатити за 1 USD
@PrePersist	JPA callback: виконується перед першим збереженням сутності
@PreUpdate	JPA callback: виконується перед кожним оновленням сутності

15. Повні лістинги файлів

У цьому розділі наведено повний вихідний код усіх файлів проекту у порядку від точки входу до сервісів.

1. src/main/java/ua/edu/example/pricetracker/PriceTrackerApplication.java

```
package ua.edu.example.pricetracker;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PriceTrackerApplication {

    public static void main(String[] args) {
        SpringApplication.run(PriceTrackerApplication.class, args);
    }
}
```

2. src/main/java/ua/edu/example/pricetracker/config/AppConfig.java

```
package ua.edu.example.pricetracker.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

3. src/main/java/ua/edu/example/pricetracker/controller/HomeController.java

```
package ua.edu.example.pricetracker.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import ua.edu.example.pricetracker.service.ProductService;
```

```
@Controller
public class HomeController {

    private final ProductService productService;

    public HomeController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/")
    public String index(Model model) {
        model.addAttribute("products", productService.getProducts(null));
        return "index";
    }
}
```

4. src/main/java/ua/edu/example/pricetracker/controller/ProductController.java

```
package ua.edu.example.pricetracker.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import ua.edu.example.pricetracker.dto.ProductCreateRequest;
import ua.edu.example.pricetracker.dto.ProductDTO;
import ua.edu.example.pricetracker.service.ProductService;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Controller
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/products")
    public String productsPage(@RequestParam(value = "sort", required = false)
String sort,
                               Model model) {
        model.addAttribute("products", productService.getProducts(sort));
        model.addAttribute("currentSort", sort);
        return "products";
    }

    @PostMapping("/api/products")
    @ResponseBody
    public ResponseEntity<Map<String, Object>> addProduct(
            @RequestBody ProductCreateRequest request) {
        Map<String, Object> response = new HashMap<>();
        try {
            ProductDTO product = productService.addProduct(request.getUrl());
            response.put("success", true);
```

```
            response.put("message", "Товар успішно додано");
            response.put("product", product);
            return ResponseEntity.ok(response);
        } catch (IllegalArgumentException e) {
            response.put("success", false);
            response.put("message", e.getMessage());
            return ResponseEntity.badRequest().body(response);
        } catch (Exception e) {
            response.put("success", false);
            response.put("message", e.getMessage() != null ? e.getMessage()
                    : "Не вдалося додати товар");
            return ResponseEntity.internalServerError().body(response);
        }
    }

    @PutMapping("/api/products/{id}/refresh")
    @ResponseBody
    public ResponseEntity<Map<String, Object>> refreshProduct(@PathVariable Long
id) {
        Map<String, Object> response = new HashMap<>();
        try {
            ProductDTO product = productService.refreshProduct(id);
            response.put("success", true);
            response.put("message", "Ціну оновлено!");
            response.put("product", product);
            return ResponseEntity.ok(response);
        } catch (Exception e) {
            response.put("success", false);
            response.put("message", e.getMessage() != null ? e.getMessage()
                    : "Не вдалося оновити ціну");
            return ResponseEntity.internalServerError().body(response);
        }
    }

    @DeleteMapping("/api/products/{id}")
    @ResponseBody
    public ResponseEntity<Map<String, Object>> deleteProduct(@PathVariable Long
id) {
        Map<String, Object> response = new HashMap<>();
        try {
            productService.deleteProduct(id);
            response.put("success", true);
            response.put("message", "Товар видалено");
            return ResponseEntity.ok(response);
        } catch (Exception e) {
            response.put("success", false);
            response.put("message", e.getMessage() != null ? e.getMessage()
                    : "Не вдалося видалити товар");
            return ResponseEntity.internalServerError().body(response);
        }
    }

    @GetMapping("/api/products")
    @ResponseBody
    public ResponseEntity<List<ProductDTO>> getProducts(
            @RequestParam(value = "sort", required = false) String sort) {
        return ResponseEntity.ok(productService.getProducts(sort));
    }
}
```

5. `src/main/java/ua/edu/example/pricetracker/dto/ProductCreateRequest.java`

```java
package ua.edu.example.pricetracker.dto;

public class ProductCreateRequest {

    private String url;

    public ProductCreateRequest() {}

    public ProductCreateRequest(String url) {
        this.url = url;
    }

    public String getUrl() { return url; }
    public void setUrl(String url) { this.url = url; }
}
```

6. src/main/java/ua/edu/example/pricetracker/dto/ProductDTO.java

```java
package ua.edu.example.pricetracker.dto;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public class ProductDTO {

    private Long id;
    private String name;
    private String productUrl;
    private String imageUrl;
    private BigDecimal priceUah;
    private BigDecimal priceUsd;
    private boolean inStock;
    private LocalDateTime updatedAt;

    public ProductDTO() {}

    public ProductDTO(Long id, String name, String productUrl, String imageUrl,
                      BigDecimal priceUah, BigDecimal priceUsd, boolean inStock,
                      LocalDateTime updatedAt) {
        this.id = id;
        this.name = name;
        this.productUrl = productUrl;
        this.imageUrl = imageUrl;
        this.priceUah = priceUah;
        this.priceUsd = priceUsd;
        this.inStock = inStock;
        this.updatedAt = updatedAt;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getProductUrl() { return productUrl; }
    public void setProductUrl(String productUrl) { this.productUrl = productUrl; }
    public String getImageUrl() { return imageUrl; }
    public void setImageUrl(String imageUrl) { this.imageUrl = imageUrl; }
    public BigDecimal getPriceUah() { return priceUah; }
    public void setPriceUah(BigDecimal priceUah) { this.priceUah = priceUah; }
    public BigDecimal getPriceUsd() { return priceUsd; }
```

```
    public void setPriceUsd(BigDecimal priceUsd) { this.priceUsd = priceUsd; }
    public boolean isInStock() { return inStock; }
    public void setInStock(boolean inStock) { this.inStock = inStock; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
    public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt =
updatedAt; }
}
```

7. src/main/java/ua/edu/example/pricetracker/dto/ScrapedProductData.java

```
package ua.edu.example.pricetracker.dto;

import java.math.BigDecimal;

public class ScrapedProductData {

    private String name;
    private String imageUrl;
    private BigDecimal priceUah;
    private boolean inStock;

    public ScrapedProductData() {}

    public ScrapedProductData(String name, String imageUrl,
                              BigDecimal priceUah, boolean inStock) {
        this.name = name;
        this.imageUrl = imageUrl;
        this.priceUah = priceUah;
        this.inStock = inStock;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getImageUrl() { return imageUrl; }
    public void setImageUrl(String imageUrl) { this.imageUrl = imageUrl; }
    public BigDecimal getPriceUah() { return priceUah; }
    public void setPriceUah(BigDecimal priceUah) { this.priceUah = priceUah; }
    public boolean isInStock() { return inStock; }
    public void setInStock(boolean inStock) { this.inStock = inStock; }
}
```

8. src/main/java/ua/edu/example/pricetracker/model/Product.java

```
package ua.edu.example.pricetracker.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    @Column(name = "name", nullable = false, length = 500)
    private String name;

    @Column(name = "product_url", nullable = false, unique = true, length = 1000)
    private String productUrl;

    @Column(name = "image_url", length = 1000)
    private String imageUrl;

    @Column(name = "price_uah", nullable = false, precision = 10, scale = 2)
    private BigDecimal priceUah;

    @Column(name = "price_usd", nullable = false, precision = 10, scale = 2)
    private BigDecimal priceUsd;

    @Column(name = "in_stock", nullable = false)
    private boolean inStock = true;

    @Column(name = "is_deleted", nullable = false)
    private boolean isDeleted = false;

    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getProductUrl() { return productUrl; }
    public void setProductUrl(String productUrl) { this.productUrl = productUrl; }
    public String getImageUrl() { return imageUrl; }
    public void setImageUrl(String imageUrl) { this.imageUrl = imageUrl; }
    public BigDecimal getPriceUah() { return priceUah; }
    public void setPriceUah(BigDecimal priceUah) { this.priceUah = priceUah; }
    public BigDecimal getPriceUsd() { return priceUsd; }
    public void setPriceUsd(BigDecimal priceUsd) { this.priceUsd = priceUsd; }
    public boolean isInStock() { return inStock; }
    public void setInStock(boolean inStock) { this.inStock = inStock; }
    public boolean isDeleted() { return isDeleted; }
    public void setDeleted(boolean deleted) { isDeleted = deleted; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt =
createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
    public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt =
updatedAt; }
```

```
}
```

9.
src/main/java/ua/edu/example/pricetracker/model/PriceHistory.java

```
package ua.edu.example.pricetracker.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "price_history")
public class PriceHistory {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(name = "price_uah", nullable = false, precision = 10, scale = 2)
    private BigDecimal priceUah;

    @Column(name = "price_usd", nullable = false, precision = 10, scale = 2)
    private BigDecimal priceUsd;

    @Column(name = "recorded_at", nullable = false)
    private LocalDateTime recordedAt;

    @PrePersist
    protected void onCreate() { recordedAt = LocalDateTime.now(); }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Product getProduct() { return product; }
    public void setProduct(Product product) { this.product = product; }
    public BigDecimal getPriceUah() { return priceUah; }
    public void setPriceUah(BigDecimal priceUah) { this.priceUah = priceUah; }
    public BigDecimal getPriceUsd() { return priceUsd; }
    public void setPriceUsd(BigDecimal priceUsd) { this.priceUsd = priceUsd; }
    public LocalDateTime getRecordedAt() { return recordedAt; }
    public void setRecordedAt(LocalDateTime r) { this.recordedAt = r; }
}
```

10.
src/main/java/ua/edu/example/pricetracker/model/ExchangeRate.java

```
package ua.edu.example.pricetracker.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;
```

```
@Entity
@Table(name = "exchange_rates")
public class ExchangeRate {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "currency_code", nullable = false, length = 3)
    private String currencyCode;

    @Column(name = "buy_rate", nullable = false, precision = 10, scale = 4)
    private BigDecimal buyRate;

    @Column(name = "sell_rate", nullable = false, precision = 10, scale = 4)
    private BigDecimal sellRate;

    @Column(name = "fetched_at", nullable = false)
    private LocalDateTime fetchedAt;

    @PrePersist
    protected void onCreate() { fetchedAt = LocalDateTime.now(); }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getCurrencyCode() { return currencyCode; }
    public void setCurrencyCode(String currencyCode) { this.currencyCode =
currencyCode; }
    public BigDecimal getBuyRate() { return buyRate; }
    public void setBuyRate(BigDecimal buyRate) { this.buyRate = buyRate; }
    public BigDecimal getSellRate() { return sellRate; }
    public void setSellRate(BigDecimal sellRate) { this.sellRate = sellRate; }
    public LocalDateTime getFetchedAt() { return fetchedAt; }
    public void setFetchedAt(LocalDateTime fetchedAt) { this.fetchedAt =
fetchedAt; }
}
```

11. src/main/java/ua/edu/example/pricetracker/repository/ProductRepository.java

```
package ua.edu.example.pricetracker.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import ua.edu.example.pricetracker.model.Product;
import java.util.List;
import java.util.Optional;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    Optional<Product> findByProductUrl(String productUrl);

    List<Product> findByIsDeletedFalse();

    List<Product> findByIsDeletedFalseOrderByPriceUahAsc();

    List<Product> findByIsDeletedFalseOrderByPriceUahDesc();
```

```
}
```

12. src/main/java/ua/edu/example/pricetracker/repository/PriceHistoryRepository.java

```
package ua.edu.example.pricetracker.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import ua.edu.example.pricetracker.model.PriceHistory;
import java.util.List;

@Repository
public interface PriceHistoryRepository extends JpaRepository<PriceHistory, Long>
{

    List<PriceHistory> findByProductIdOrderByRecordedAtDesc(Long productId);
}
```

13. src/main/java/ua/edu/example/pricetracker/repository/ExchangeRateRepository.java

```
package ua.edu.example.pricetracker.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import ua.edu.example.pricetracker.model.ExchangeRate;
import java.util.Optional;

@Repository
public interface ExchangeRateRepository extends JpaRepository<ExchangeRate, Long>
{

    Optional<ExchangeRate> findTopByCurrencyCodeOrderByFetchedAtDesc(
            String currencyCode);
}
```

14. src/main/java/ua/edu/example/pricetracker/service/ProductService.java

```
package ua.edu.example.pricetracker.service;

import org.springframework.stereotype.Service;
import ua.edu.example.pricetracker.dto.ProductDTO;
import ua.edu.example.pricetracker.dto.ScrapedProductData;
import ua.edu.example.pricetracker.model.Product;
import ua.edu.example.pricetracker.repository.ProductRepository;
import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;
```

```
@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final ScraperService scraperService;
    private final ExchangeRateService exchangeRateService;
    private final PriceHistoryService priceHistoryService;

    public ProductService(ProductRepository productRepository,
                          ScraperService scraperService,
                          ExchangeRateService exchangeRateService,
                          PriceHistoryService priceHistoryService) {
        this.productRepository = productRepository;
        this.scraperService = scraperService;
        this.exchangeRateService = exchangeRateService;
        this.priceHistoryService = priceHistoryService;
    }

    public ProductDTO addProduct(String url) {
        if (url != null) url = url.trim();

        Optional<Product> existing = productRepository.findByProductUrl(url);
        if (existing.isPresent()) {
            Product p = existing.get();
            if (!p.isDeleted()) {
                throw new IllegalArgumentException("Цей товар вже додано до
списку");
            }
            p.setDeleted(false);
            return refreshProductInternal(p);
        }

        ScrapedProductData scraped = scraperService.scrapeProduct(url);
        BigDecimal priceUsd =
exchangeRateService.convertUahToUsd(scraped.getPriceUah());

        Product product = new Product();
        product.setName(scraped.getName());
        product.setProductUrl(url);
        product.setImageUrl(scraped.getImageUrl());
        product.setPriceUah(scraped.getPriceUah());
        product.setPriceUsd(priceUsd);
        product.setInStock(scraped.isInStock());
        product = productRepository.save(product);

        priceHistoryService.recordPrice(product, scraped.getPriceUah(), priceUsd);
        return toDTO(product);
    }

    public ProductDTO refreshProduct(Long id) {
        Product product = productRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Товар не знайдено"));
        if (product.isDeleted()) throw new RuntimeException("Товар було
видалено");
        return refreshProductInternal(product);
    }

    public void deleteProduct(Long id) {
        Product product = productRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Товар не знайдено"));
        product.setDeleted(true);
        productRepository.save(product);
```

```
    }

    public List<ProductDTO> getProducts(String sort) {
        List<Product> products;
        if ("asc".equalsIgnoreCase(sort))
            products = productRepository.findByIsDeletedFalseOrderByPriceUahAsc();
        else if ("desc".equalsIgnoreCase(sort))
            products =
productRepository.findByIsDeletedFalseOrderByPriceUahDesc();
        else
            products = productRepository.findByIsDeletedFalse();
        return products.stream().map(this::toDTO).collect(Collectors.toList());
    }

    private ProductDTO refreshProductInternal(Product product) {
        ScrapedProductData scraped =
scraperService.scrapeProduct(product.getProductUrl());
        BigDecimal priceUsd =
exchangeRateService.convertUahToUsd(scraped.getPriceUah());

        boolean priceChanged = product.getPriceUah() == null ||
            product.getPriceUah().compareTo(scraped.getPriceUah()) != 0 ||
            product.getPriceUsd() == null ||
            product.getPriceUsd().compareTo(priceUsd) != 0;

        product.setName(scraped.getName());
        product.setImageUrl(scraped.getImageUrl());
        product.setPriceUah(scraped.getPriceUah());
        product.setPriceUsd(priceUsd);
        product.setInStock(scraped.isInStock());
        product = productRepository.save(product);

        if (priceChanged) {
            priceHistoryService.recordPrice(product, scraped.getPriceUah(),
priceUsd);
        }
        return toDTO(product);
    }

    private ProductDTO toDTO(Product p) {
        return new ProductDTO(p.getId(), p.getName(), p.getProductUrl(),
                p.getImageUrl(), p.getPriceUah(), p.getPriceUsd(),
                p.isInStock(), p.getUpdatedAt());
    }
}
```

15. src/main/java/ua/edu/example/pricetracker/service/ScraperService.java

```
package ua.edu.example.pricetracker.service;

import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.springframework.stereotype.Service;
import ua.edu.example.pricetracker.dto.ScrapedProductData;
import java.io.IOException;
import java.math.BigDecimal;
```

```java
@Service
public class ScraperService {

    public ScrapedProductData scrapeProduct(String url) {
        validateUrl(url);
        try {
            Document doc = Jsoup.connect(url)
                    .userAgent("Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                            + "AppleWebKit/537.36 (KHTML, like Gecko) "
                            + "Chrome/120.0.0.0 Safari/537.36")
                    .timeout(10000)
                    .get();

            Element titleElement = doc.selectFirst("h1.inbreadcrumb");
            if (titleElement == null) {
                throw new RuntimeException("Не вдалося отримати дані про товар");
            }
            String name = titleElement.text().trim();

            String imageUrl = null;
            Element imgElement = doc.selectFirst("img#mainImage");
            if (imgElement != null) {
                imageUrl = imgElement.absUrl("src");
                if (imageUrl.isEmpty()) imageUrl = imgElement.attr("src");
            }

            BigDecimal priceUah;
            Element priceMeta = doc.selectFirst("meta[itemprop=price]");
            if (priceMeta != null) {
                priceUah = new BigDecimal(priceMeta.attr("content").trim());
            } else {
                Element priceSpan = doc.selectFirst("span.update_price_options");
                if (priceSpan != null) {
                    String priceText = priceSpan.text()
                            .replaceAll("[^\\d.,]", "").replace(",", ".").trim();
                    priceUah = new BigDecimal(priceText);
                } else {
                    throw new RuntimeException("Не вдалося отримати ціну товару");
                }
            }

            boolean inStock = false;
            Element stockElement = doc.selectFirst("div.nalich");
            if (stockElement != null) {
                String stockText = stockElement.text().toLowerCase();
                inStock = stockText.contains("є в наявності")
                        || stockText.contains("есть в наличии");
            }

            return new ScrapedProductData(name, imageUrl, priceUah, inStock);

        } catch (IOException e) {
            throw new RuntimeException(
                    "Товар не знайдено за вказаним посиланням", e);
        }
    }

    private void validateUrl(String url) {
        if (url == null || url.isBlank()) {
            throw new IllegalArgumentException(
                    "Будь ласка, введіть коректне посилання");
        }
        if (!url.startsWith("https://ks-market.com.ua/ua/")) {
            throw new IllegalArgumentException(
```

```
                    "Підтримуються лише посилання формату https://ks-
market.com.ua/ua/...");
        }
    }
}
```

16. src/main/java/ua/edu/example/pricetracker/service/ExchangeRateService.java

```
package ua.edu.example.pricetracker.service;

import com.fasterxml.jackson.annotation.JsonProperty;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import ua.edu.example.pricetracker.model.ExchangeRate;
import ua.edu.example.pricetracker.repository.ExchangeRateRepository;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDateTime;
import java.util.Optional;

@Service
public class ExchangeRateService {

    private static final String PRIVATBANK_API_URL =
            "https://api.privatbank.ua/p24api/pubinfo?exchange&coursid=5";
    private static final String USD = "USD";
    private static final long CACHE_HOURS = 1;

    private final ExchangeRateRepository exchangeRateRepository;
    private final RestTemplate restTemplate;

    public ExchangeRateService(ExchangeRateRepository exchangeRateRepository,
                               RestTemplate restTemplate) {
        this.exchangeRateRepository = exchangeRateRepository;
        this.restTemplate = restTemplate;
    }

    public BigDecimal getUsdSellRate() {
        Optional<ExchangeRate> cached = exchangeRateRepository
                .findTopByCurrencyCodeOrderByFetchedAtDesc(USD);

        if (cached.isPresent() && cached.get().getFetchedAt()
                .isAfter(LocalDateTime.now().minusHours(CACHE_HOURS))) {
            return cached.get().getSellRate();
        }
        return fetchAndCacheRate();
    }

    public BigDecimal convertUahToUsd(BigDecimal priceUah) {
        BigDecimal sellRate = getUsdSellRate();
        return priceUah.divide(sellRate, 2, RoundingMode.HALF_UP);
    }

    private BigDecimal fetchAndCacheRate() {
        try {
            PrivatBankRate[] rates = restTemplate.getForObject(
                    PRIVATBANK_API_URL, PrivatBankRate[].class);
            if (rates != null) {
```

```
                for (PrivatBankRate rate : rates) {
                    if (USD.equals(rate.ccy)) {
                        ExchangeRate exchangeRate = new ExchangeRate();
                        exchangeRate.setCurrencyCode(USD);
                        exchangeRate.setBuyRate(new BigDecimal(rate.buy));
                        exchangeRate.setSellRate(new BigDecimal(rate.sale));
                        exchangeRateRepository.save(exchangeRate);
                        return new BigDecimal(rate.sale);
                    }
                }
            }
            throw new RuntimeException("Не вдалося отримати курс USD");
        } catch (Exception e) {
            Optional<ExchangeRate> anyCached = exchangeRateRepository
                    .findTopByCurrencyCodeOrderByFetchedAtDesc(USD);
            if (anyCached.isPresent()) return anyCached.get().getSellRate();
            throw new RuntimeException("Не вдалося отримати курс валюти", e);
        }
    }

    private static class PrivatBankRate {
        @JsonProperty("ccy")      public String ccy;
        @JsonProperty("base_ccy") public String baseCcy;
        @JsonProperty("buy")      public String buy;
        @JsonProperty("sale")     public String sale;
    }
}
```

17. src/main/java/ua/edu/example/pricetracker/service/PriceHistoryService.java

```
package ua.edu.example.pricetracker.service;

import org.springframework.stereotype.Service;
import ua.edu.example.pricetracker.model.PriceHistory;
import ua.edu.example.pricetracker.model.Product;
import ua.edu.example.pricetracker.repository.PriceHistoryRepository;
import java.math.BigDecimal;
import java.util.List;

@Service
public class PriceHistoryService {

    private final PriceHistoryRepository priceHistoryRepository;

    public PriceHistoryService(PriceHistoryRepository priceHistoryRepository) {
        this.priceHistoryRepository = priceHistoryRepository;
    }

    public void recordPrice(Product product,
                            BigDecimal priceUah, BigDecimal priceUsd) {
        PriceHistory history = new PriceHistory();
        history.setProduct(product);
        history.setPriceUah(priceUah);
        history.setPriceUsd(priceUsd);
        priceHistoryRepository.save(history);
    }

    public List<PriceHistory> getHistoryForProduct(Long productId) {
        return priceHistoryRepository
```

```
                    .findByProductIdOrderByRecordedAtDesc(productId);
    }
}
```